

\$ git init mastery

Git & GitHub

Complete Beginner-to-Advanced Guide

From version control fundamentals to professional team workflows, CI/CD, security practices, and GitHub mastery.

Git Internals

GitHub
Workflow

Security

CI/CD

18
Sections

Table of Contents

Section 1	What is Git?	1
Section 2	Installing Git	6
Section 3	Creating Your First Repository	9
Section 4	Git Workflow Basics	13
Section 5	Understanding Commits	17
Section 6	Branching	22
Section 7	Merging and Rebasing	28
Section 8	GitHub Explained	35
Section 9	Connecting Local Repo to GitHub	39
Section 10	Professional GitHub Workflow	45
Section 11	Important Git Commands	52
Section 12	Git Internals	62
Section 13	Common Beginner Mistakes	69
Section 14	Security Best Practices	74
Section 15	Real Project Example	80
Section 16	GitHub Portfolio Tips	87
Section 17	Advanced Topics	92
Section 18	Cheat Sheet	100

Section 1 — What is Git?

Before writing a single Git command, you need to understand the **problem Git was built to solve**. Every professional developer has been in this situation: you have a working piece of code, you make changes, something breaks, and you cannot remember exactly what you changed. Or you are working with a team and two people modify the same file at the same time — who wins? Whose version is correct?

1.1 The Problem Git Solves

Imagine you are building a web application. Over time, you accumulate a collection of files like:

```
app_v1.py
app_v2.py
app_v2_final.py
app_v2_final_REAL.py
app_v2_final_REAL_v3.py
```

This is what developers did before version control systems. The result is chaos — you do not know which file is the latest, what changed between versions, or how to safely experiment without destroying working code.

Git solves this by acting as a **time machine for your code**. It records every change you make with a timestamp, author, and description. You can go back to any point in time, compare any two versions, work on experimental features without risking the main codebase, and collaborate with other developers without overwriting each other's work.

■ ■ Note

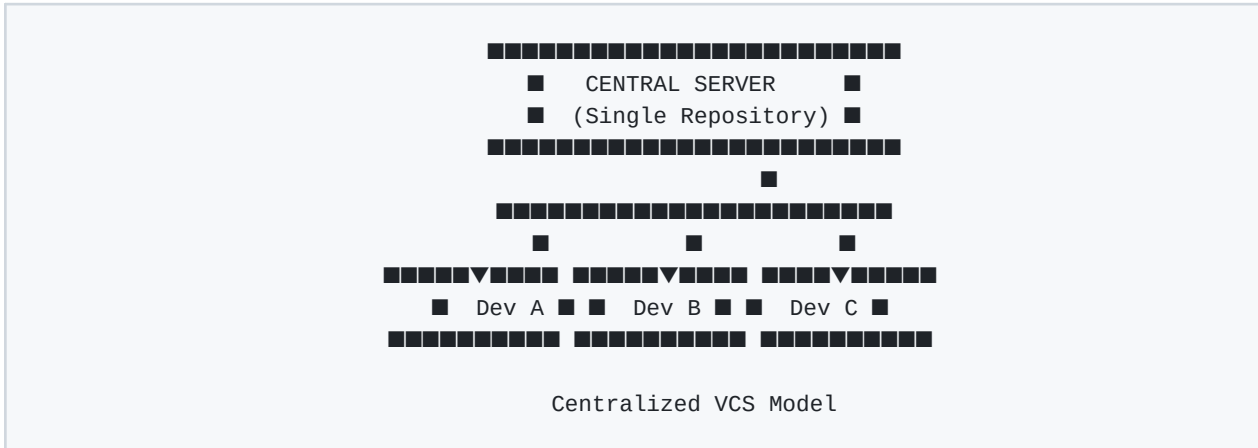
Git does not track files — it tracks **changes to files**. This distinction is critical to understanding how Git works internally.

1.2 Version Control Systems (VCS)

A **Version Control System (VCS)** is software that records changes to a file or set of files over time so that you can recall specific versions later. There are two main types:

Centralized VCS (CVCS)

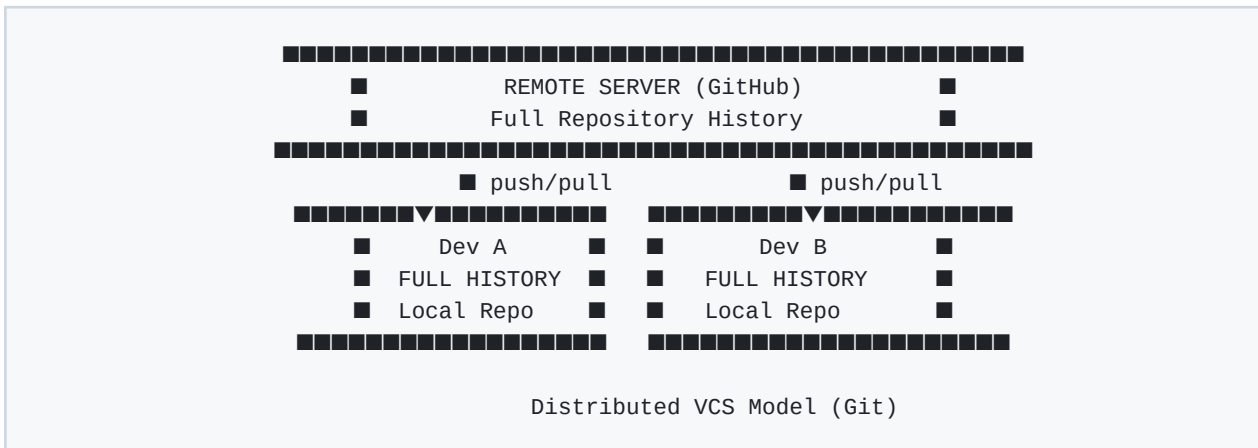
In a Centralized VCS like **SVN (Subversion)** or **CVS**, there is a single central server that holds all the versioned files. Every developer checks files out from and commits changes to this central server.



Problems with CVCS: if the central server goes down, nobody can collaborate or save versioned changes. If the central disk gets corrupted, you lose everything. Every operation requires a network connection.

Distributed VCS (DVCS)

In a Distributed VCS like **Git** or **Mercurial**, every developer has a full copy of the entire repository history on their local machine. There is no single point of failure.



1.3 Why Git Became the Industry Standard

Git was created in **2005** by **Linus Torvalds** (the creator of Linux) when the Linux kernel project needed a better version control system. Git was designed with these goals in mind:

- **Speed:** Nearly all operations are local — no network round trips required.

- **Integrity:** Every piece of data is checksummed with SHA-1 hashes before storage.
- **Non-linear development:** Thousands of branches can be created and merged efficiently.
- **Distributed:** Every clone is a full backup of the repository.
- **Open source:** Free to use, widely adopted, with massive community support.

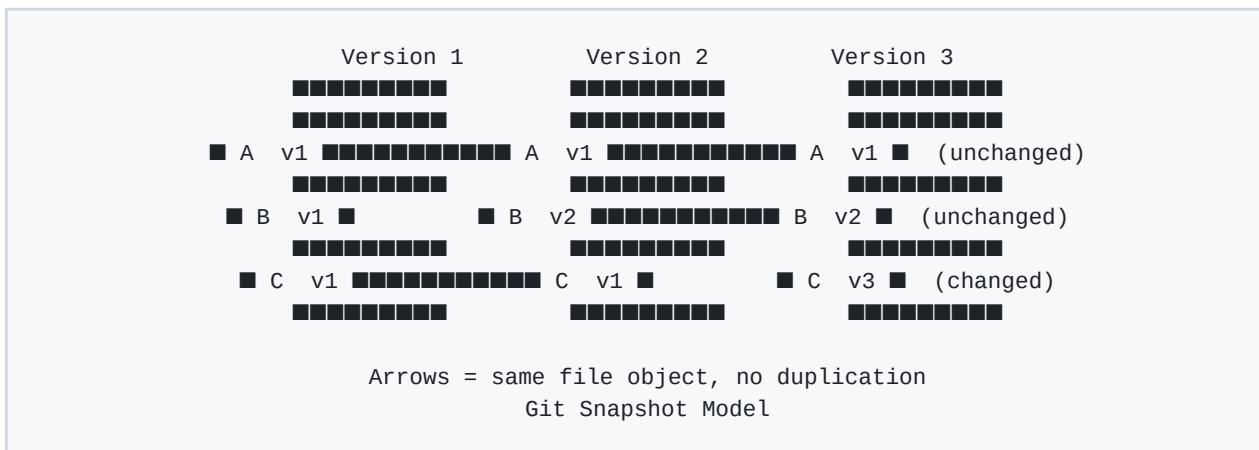
Today, Git is used by virtually every software company in the world. GitHub, which is built on top of Git, hosts over 330 million repositories. Understanding Git is a non-negotiable skill for any professional developer.

1.4 How Git Stores History Internally

This is where most tutorials fail to teach you the truth. **Git does not store diffs** (the difference between files). It stores **snapshots** of the entire project at every commit.

Snapshots, Not Diffs

When you make a commit, Git takes a snapshot of all your tracked files and stores a reference to that snapshot. If a file has not changed since the previous commit, Git simply stores a reference to the identical file it already has — it does not store the file again. This makes Git extremely efficient while still having access to the complete project state at every point in history.



SHA Hashes — The Identity of Everything in Git

Every piece of data in Git — every file, every directory, every commit — is identified by a **SHA-1 hash**: a 40-character hexadecimal string computed from the content itself.

```
# Example SHA-1 hash (commit hash):
a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5

# Git computes this from the content using:
```

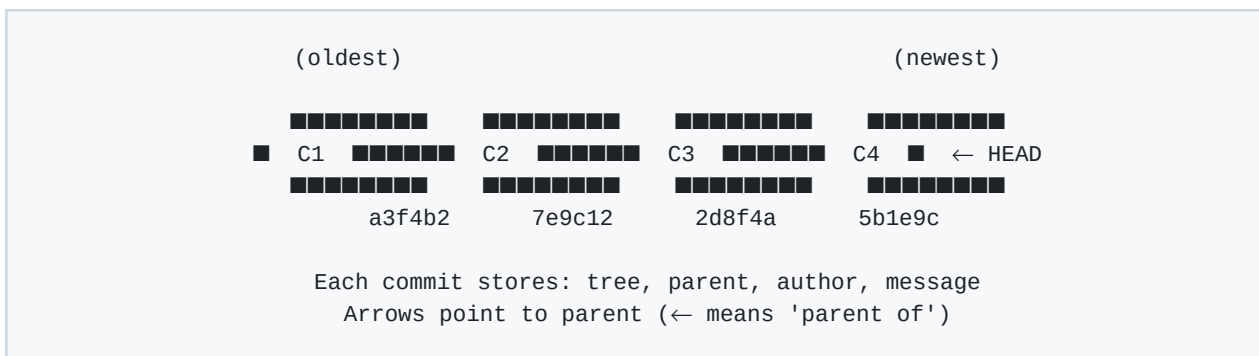
```
# SHA1(content) → unique fingerprint

# You can see this with:
$ git cat-file -t a3f4b2c9 # Shows the type
$ git cat-file -p a3f4b2c9 # Shows the content
```

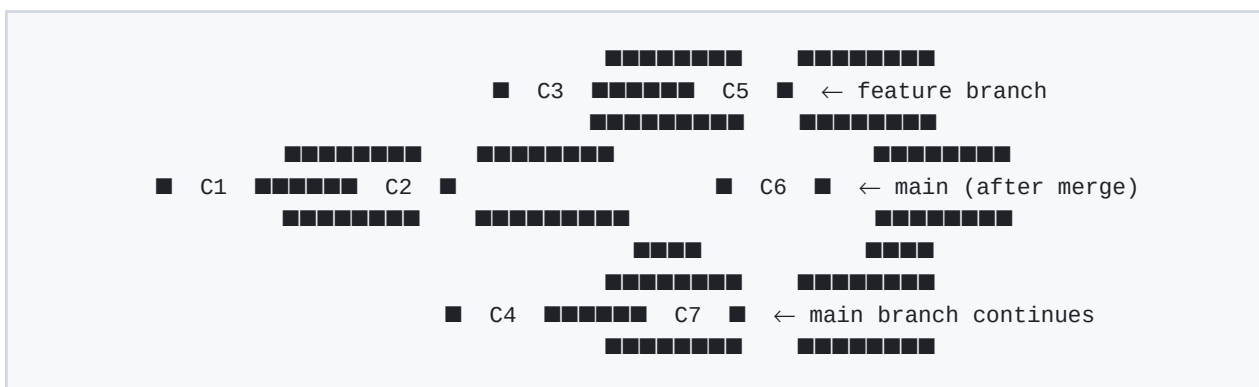
The SHA hash is a **cryptographic guarantee**. If even a single character changes in a file, the hash changes completely. This means it is practically impossible to corrupt Git history without detection — every object in Git is verified by its hash.

1.5 The Commit Graph

Git history is structured as a **Directed Acyclic Graph (DAG)**. Each commit points to its parent commit(s), forming a chain of history:



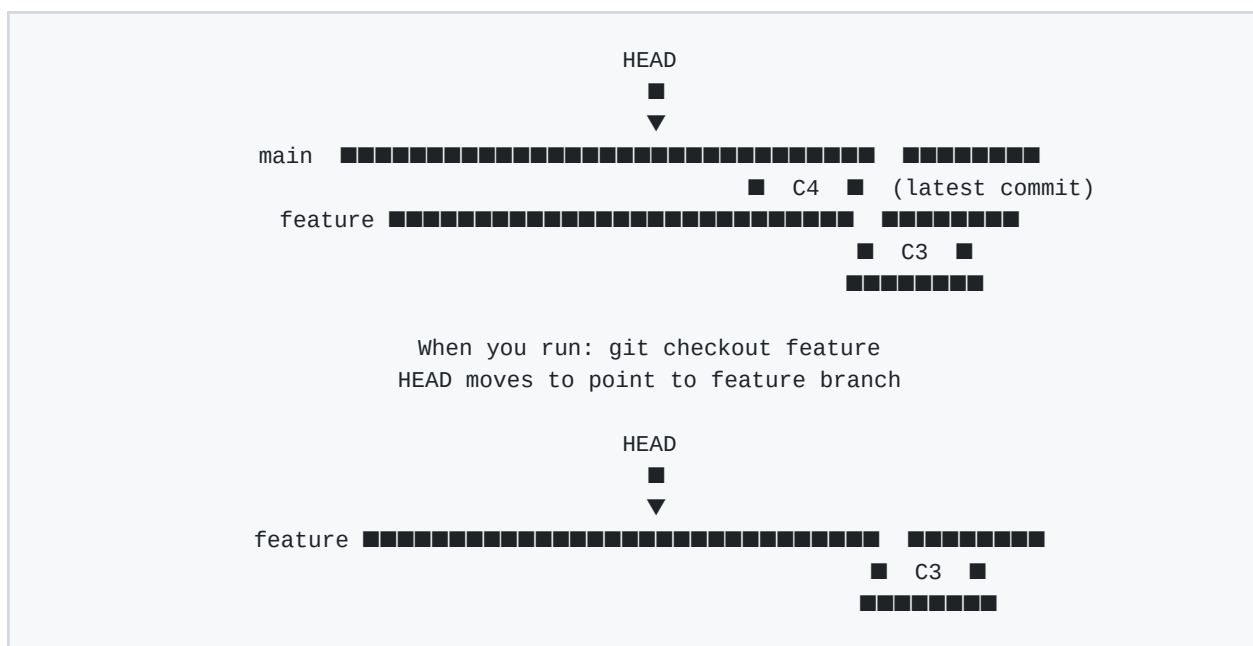
When branches exist, the graph splits and merges:



1.6 Branches and HEAD — Visual Explanation

A **branch** in Git is simply a lightweight pointer — a text file containing a SHA hash — pointing to a specific commit. When you create a branch, Git creates a new pointer; it does not copy files.

HEAD is a special pointer that tells Git: *'this is where I currently am.'* HEAD usually points to a branch name, and the branch name points to a commit.



■ Tip

Understanding that branches are just pointers is the key to understanding Git. Creating a branch is $O(1)$ — it takes the same time whether you have 10 commits or 10 million.

Section 2 — Installing Git

Before you can use Git, you need to install it on your system and configure your identity. Git needs to know who you are because every commit you make is stamped with your name and email — this is part of the immutable history record.

2.1 Installing on Windows

The recommended way to install Git on Windows is using the official installer from git-scm.com. This installer includes **Git Bash**, a terminal emulator that gives you a Unix-like environment.

- Step 1: Download from <https://git-scm.com/download/win>
- Step 2: Run the installer. The default options are appropriate for most developers.
- Step 3: Important installer choices:
 - Use Git from Git Bash only (recommended for beginners)
 - Use OpenSSH (recommended)
 - Use the OpenSSL library
 - Checkout Windows-style, commit Unix-style line endings

```
# Alternative: Install via winget (Windows Package Manager)
winget install --id Git.Git -e --source winget

# Or via Chocolatey:
choco install git

# Verify installation in Git Bash or PowerShell:
$ git --version
git version 2.43.0.windows.1
```

2.2 Installing on Linux

On Linux, Git is available through the package manager. Most distributions include Git by default or have it available in their repositories.

```
# Ubuntu / Debian
$ sudo apt update
```

```
$ sudo apt install git

# Fedora / RHEL / CentOS
$ sudo dnf install git

# Arch Linux
$ sudo pacman -S git

# Alpine Linux
$ sudo apk add git

# Verify:
$ git --version
git version 2.43.0
```

2.3 Installing on macOS

On macOS, the easiest method is using Homebrew. If you do not have Homebrew, you can also trigger the Xcode Command Line Tools installation which includes Git.

```
# Method 1: Homebrew (recommended)
$ brew install git

# Method 2: Trigger Xcode Command Line Tools
$ git --version
# This will prompt macOS to install Developer Tools if not present

# Method 3: Download from git-scm.com
# https://git-scm.com/download/mac

# Verify:
$ git --version
git version 2.43.0

# macOS ships with an older Apple Git – Homebrew gives you the latest
$ which git
/usr/local/bin/git      # Homebrew version
# vs
/usr/bin/git            # Apple's older version
```

2.4 First-Time Setup

After installing Git, the first thing you must do is set your identity. This is stored in every commit you create — it is part of the public record if you push to GitHub.

```
# Set your name (used in all commits)
$ git config --global user.name "Your Name"

# Set your email (used in all commits)
$ git config --global user.email "you@example.com"

# Set your default editor (nano, vim, code, etc.)
$ git config --global core.editor "code --wait"

# Set default branch name to 'main' (modern convention)
$ git config --global init.defaultBranch main

# Enable colored output
$ git config --global color.ui auto

# Verify all config:
$ git config --list
user.name=Your Name
user.email=you@example.com
core.editor=code --wait
init.defaultbranch=main
color.ui=auto
```

Global vs Local Config

Git configuration has three levels, each overriding the one above it:

Level	File Location	Scope	Use Case
--system	/etc/gitconfig	All users on the machine	System-wide defaults
--global	~/.gitconfig	Your user account	Your personal settings
--local	.git/config	One repository	Project-specific overrides

A real-world use case: you might have a work email globally configured, but a personal email for open-source projects. You would set the personal email locally inside each open-source repo:

```
# Inside an open-source project folder:
$ git config --local user.email "personal@gmail.com"

# This only affects THIS repository
```

```
# Your company email is still the global default
```

```
# Check what config applies to the current repo:
```

```
$ git config user.email
```

```
personal@gmail.com
```

```
# See where each value comes from:
```

```
$ git config --list --show-origin
```

Section 3 — Creating Your First Repository

Now that Git is installed and configured, you will create your first repository and understand exactly what happens at each step — not just the commands, but the internal mechanics.

3.1 Creating a Project

```
# Create a new directory for your project
$ mkdir my-project

# Navigate into it
$ cd my-project

# Initialize an empty Git repository
$ git init
Initialized empty Git repository in /home/user/my-project/.git/
```

3.2 What .git Contains — The Repository Brain

The **.git** folder is the entire Git repository. If you delete it, Git history is gone but your working files remain. If you copy it to another machine, you have the complete repository. Let's explore what's inside:

```
$ ls -la .git/
total 40
drwxr-xr-x  8 user group 256 Jan 15 10:22 .
drwxr-xr-x  3 user group  96 Jan 15 10:22 ..
-rw-r--r--  1 user group  23 Jan 15 10:22 HEAD
drwxr-xr-x  2 user group  64 Jan 15 10:22 branches/
-rw-r--r--  1 user group  92 Jan 15 10:22 config
-rw-r--r--  1 user group  73 Jan 15 10:22 description
drwxr-xr-x 14 user group 448 Jan 15 10:22 hooks/
drwxr-xr-x  3 user group  96 Jan 15 10:22 info/
drwxr-xr-x  4 user group 128 Jan 15 10:22 objects/
```

```
drwxr-xr-x  4 user group 128 Jan 15 10:22 refs/
```

File/Folder	Purpose
HEAD	Points to the current branch (e.g., ref: refs/heads/main)
config	Repository-specific configuration (local settings)
description	Used by GitWeb — you rarely need this
hooks/	Scripts that run automatically on Git events (pre-commit, post-push, etc.)
info/	Contains exclude file — alternative to .gitignore for repo-specific excludes
objects/	The object database — where ALL your data is stored (blobs, trees, commits)
refs/	References — branch pointers and tag pointers

The HEAD file

```
$ cat .git/HEAD
ref: refs/heads/main

# After making the first commit, refs/heads/main will contain:
$ cat .git/refs/heads/main
a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5

# HEAD → branch name → commit SHA
# This chain is how Git knows where you are
```

3.3 The Objects Database

Git stores all data as compressed objects in **.git/objects/**. After your first commit, you would see a structure like:

```
.git/objects/
  a3/
    f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5 ← commit object
  7e/
    9c12... ← tree object
  2d/
    8f4a... ← blob object (file conten
t)
  info/
```

```
■■■ pack/
```

```
# First 2 characters = folder name  
# Remaining 38 = filename  
# This avoids having too many files in one directory
```

■ ■ Note

You will learn the full details of blobs, trees, and commit objects in Section 12 (Git Internals). For now, understand that Git stores everything by content hash.

3.4 Cloning an Existing Repository

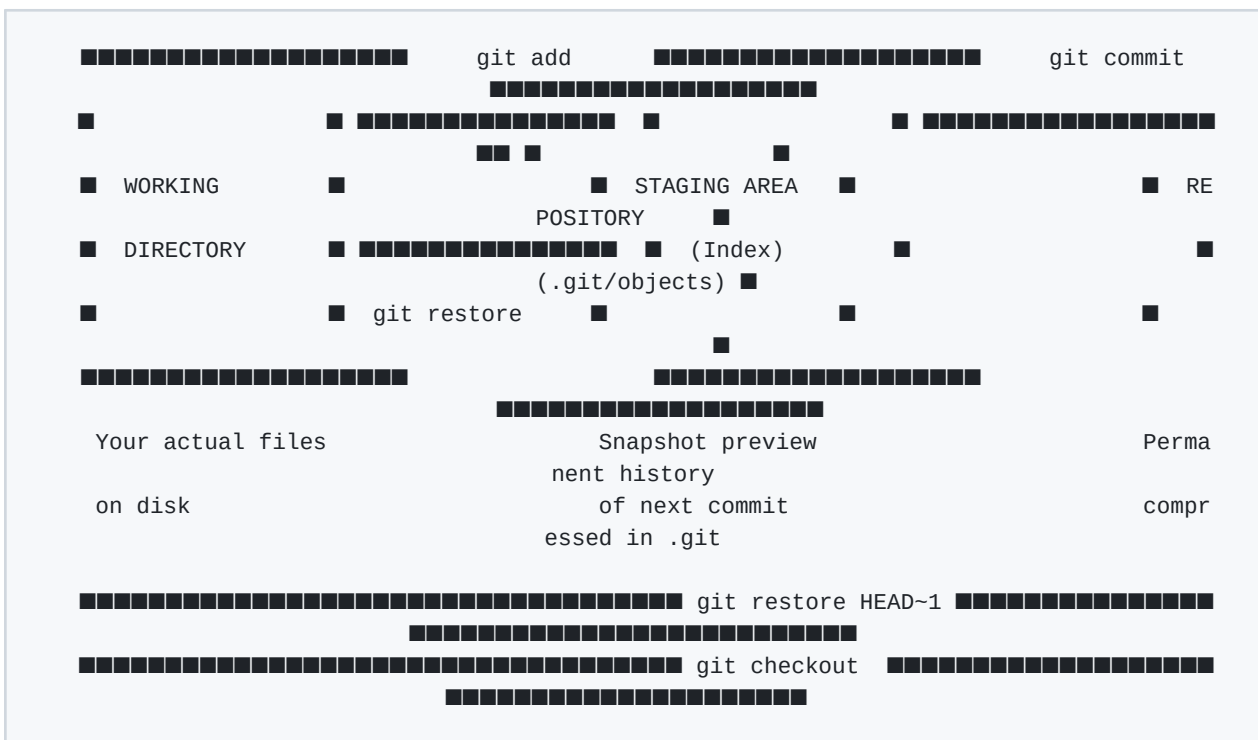
Instead of creating a new repo with **git init**, you can clone an existing one. Cloning downloads the complete repository history to your machine.

```
# Clone a repository  
$ git clone https://github.com/username/repository.git  
  
# Clone into a specific directory name  
$ git clone https://github.com/username/repository.git my-folder  
  
# Clone via SSH (requires SSH key setup)  
$ git clone git@github.com:username/repository.git  
  
# Clone only the latest snapshot (shallow clone) – faster for large repos  
$ git clone --depth=1 https://github.com/username/repository.git  
  
# What happens internally during clone:  
# 1. Creates a new directory  
# 2. Initializes .git inside it  
# 3. Downloads all objects (commits, trees, blobs)  
# 4. Sets origin remote automatically  
# 5. Checks out the default branch
```

Section 4 — Git Workflow Basics

The core Git workflow involves three areas that you must understand deeply before anything else makes sense. Every Git operation is fundamentally a movement of data between these three areas.

4.1 The Three Areas of Git



Working Directory

This is where you actually edit files. It contains the files you can see on your filesystem. Git tracks which files have been modified, added, or deleted compared to what is in the repository.

Staging Area (Index)

The **staging area** (also called the **index**) is a preparation area for your next commit. When you run **git add**, you are telling Git: 'include this change in the next commit.' This gives you precise control over what goes into each commit. You might have made 10 changes but want to commit only 3 of them — staging makes this possible.

■ ■ Note

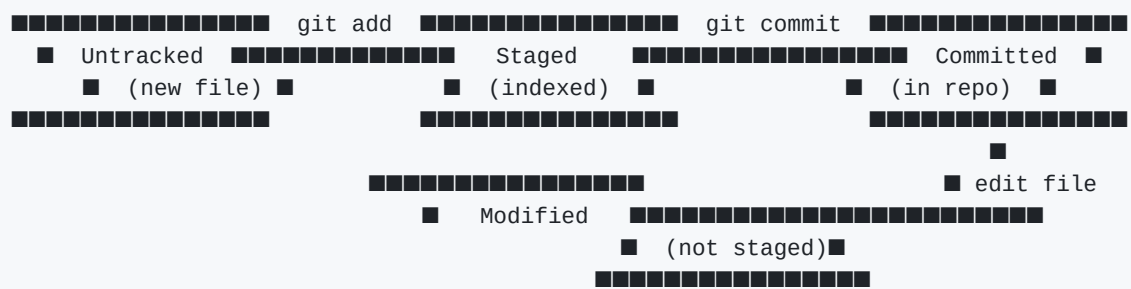
The staging area is stored in **.git/index** — a binary file. Understanding it exists as a separate concept from both your working files and your committed history is crucial.

Repository (.git)

This is the permanent, compressed history of your project. Once you commit, data goes here and becomes immutable (you can technically change history but it requires explicit commands). The repository is entirely inside the **.git** directory.

4.2 File States in Git

Any file in your working directory can be in one of these states:



File State Transitions

4.3 The Essential Commands

git status — Your Best Friend

```
$ git status
```

```
On branch main
```

```
Changes to be committed:      ← Already staged
  (use 'git restore --staged <file>' to unstage)
```

```
    new file:   index.html
    modified:   app.py
```

```
Changes not staged for commit:  ← Modified but not staged
```

```

    (use 'git add <file>' to update what will be committed)
        modified:   README.md

Untracked files:                               ← New files Git doesn't know about
    (use 'git add <file>' to include what will be committed)
        secret.env
        node_modules/

# Short status format:
$ git status -s
 M README.md      ← Modified, not staged (space M)
 M  app.py        ← Modified, staged (M space)
 A  index.html    ← New file, staged
 ?? secret.env    ← Untracked

```

git add — Moving to Staging

```

# Stage a single file
$ git add index.html

# Stage multiple files
$ git add index.html style.css app.py

# Stage an entire directory
$ git add src/

# Stage ALL changes in the working directory
$ git add .

# Interactive staging – choose which hunks to stage
$ git add -p
# This is powerful: stage only part of a file's changes

# What git add does internally:
# 1. Compresses file content into a blob object
# 2. Stores blob in .git/objects/
# 3. Updates .git/index with the new file reference
# 4. File is now 'staged'

```

git commit — Saving to History

```

# Commit with a message

```

```
$ git commit -m "Add user authentication feature"

# Opens your configured editor for a detailed message
$ git commit

# Stage all tracked files and commit in one step
$ git commit -am "Fix login validation bug"
# Warning: -a does NOT include new untracked files

# What git commit does internally:
# 1. Creates a tree object from the index
# 2. Creates a commit object with: tree hash, parent hash, author, message
# 3. Writes commit to .git/objects/
# 4. Updates the current branch pointer to the new commit SHA
# 5. Clears the index (staging area) for the committed files

# Amend the last commit (if you forgot something):
$ git commit --amend
# WARNING: This rewrites history – never amend pushed commits
```

■ ■ Warning

The **-am** flag is convenient but dangerous for beginners because it stages ALL tracked changes automatically. Always run **git status** before committing to verify exactly what goes in.

4.4 .gitignore — Excluding Files

Not every file should be tracked by Git. The **.gitignore** file tells Git which files and folders to completely ignore. This is critical for security and cleanliness.

```
# Create .gitignore in the root of your project

# Ignore all .env files (contains secrets!)
.env
.env.local
.env.production

# Ignore node_modules (huge folder, always regenerated)
node_modules/

# Ignore Python virtual environments
venv/
__pycache__/
```

```
*.pyc

# Ignore build artifacts
dist/
build/
*.o
*.exe

# Ignore OS files
.DS_Store      # macOS
Thumbs.db      # Windows

# Ignore IDE config
.idea/
.vscode/

# Ignore log files
*.log
logs/
```

■ Danger

■■ **SECURITY WARNING:** If you accidentally commit a file with secrets (API keys, passwords, tokens), simply adding it to `.gitignore` is NOT enough — the secret is already in Git history. You must use `git-filter-repo` to purge it and rotate the compromised credentials immediately.

Section 5 — Understanding Commits

Commits are the atomic units of Git history. Understanding what a commit really is — both conceptually and internally — is fundamental to becoming proficient with Git.

5.1 What a Commit Really Contains

When you run **git commit**, Git creates an object that stores more than just your file changes. A commit object contains:

```
# Inspect a commit object:
$ git cat-file -p HEAD

tree 9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e ← snapshot of files
parent a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5 ← previous commit
author Jane Smith <jane@example.com> 1705312800 +0000
committer Jane Smith <jane@example.com> 1705312800 +0000

Add user authentication with JWT tokens

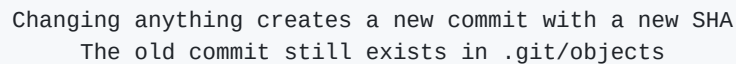
- Implement login endpoint
- Add password hashing with bcrypt
- Create JWT token generation
- Add auth middleware

Closes #42
```

Notice the structure: a commit is a pointer to a **tree** (the snapshot), a pointer to the **parent commit** (the history link), authorship metadata, and your commit message. It does NOT store diffs — it stores a reference to the complete state.

5.2 Immutable History

Once a commit is created, its SHA hash is determined by its content — including the parent hash, the tree hash, the author, and the message. If you change ANYTHING in a commit, you get a completely different SHA. This is why Git history is considered **immutable**.



When you 'amend' or 'rebase', you are not modifying commits. You are creating new commits and moving branch pointers to them. The old commits remain in the object database until garbage collection removes them.

A commit message is not just documentation for yourself — it is a communication tool for your team, your future self, and anyone who reads the history. The industry-standard convention is the **Conventional Commits** specification combined with the **50/72 rule**.

- **Subject line:** maximum 50 characters — concise summary in imperative mood
- **Blank line:** always separate subject from body
- **Body:** maximum 72 characters per line — explain WHAT and WHY, not HOW

```
docs: Documentation only
```



```

# Basic log
$ git log
commit a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5 (HEAD -> main)
Author: Jane Smith <jane@example.com>
Date:   Mon Jan 15 10:22:00 2024 +0000

    feat(auth): add JWT token refresh endpoint

# One-line format:
$ git log --oneline
a3f4b2c feat(auth): add JWT token refresh endpoint
7e9c121 fix(login): correct password hashing rounds
2d8f4ab feat(user): add profile update endpoint

# Graphical log (extremely useful):
$ git log --oneline --graph --decorate --all
* a3f4b2c (HEAD -> main) feat(auth): add JWT refresh
| * 9d2c8f1 (feature/chat) feat(chat): add WebSocket handler
|/
* 7e9c121 fix(login): correct password hashing
* 2d8f4ab feat(user): add profile update

# Filter by author:
$ git log --author="Jane"

# Filter by date:
$ git log --since="2024-01-01" --until="2024-01-31"

# Filter by message:
$ git log --grep="auth"

# Show changes in each commit:
$ git log -p

```

Section 6 — Branching

Branching is Git's killer feature. It allows multiple lines of development to coexist in the same repository without interfering with each other. Understanding branches as lightweight pointers — rather than copies of files — is the mental model that unlocks everything.

6.1 What Branches Really Are

A branch in Git is nothing more than a **text file containing a 40-character SHA hash**. That's it. Creating a branch does not copy files. It does not create a parallel filesystem. It creates a 41-byte file in `.git/refs/heads/`.

```
# Look at what a branch actually is on disk:
$ cat .git/refs/heads/main
a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5

# Create a new branch:
$ git branch feature/login

# Now there's a new file:
$ cat .git/refs/heads/feature/login
a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5

# Both branches point to the SAME commit right now
# No files were copied – this operation took microseconds
```

6.2 Branch Operations

```
# List all local branches (* = current branch)
$ git branch
* main
  feature/login
  bugfix/header-overflow

# List remote branches too
$ git branch -a
```

```

# List branches with last commit info
$ git branch -v
* main          a3f4b2c feat(auth): add JWT refresh
  feature/login a3f4b2c feat(auth): add JWT refresh

# Create a new branch (does NOT switch to it)
$ git branch feature/signup

# Switch to an existing branch
$ git switch feature/login
# or the older syntax:
$ git checkout feature/login

# Create AND switch (most common pattern):
$ git switch -c feature/new-feature
# or:
$ git checkout -b feature/new-feature

# Delete a branch (safe – refuses if unmerged)
$ git branch -d feature/login

# Force delete (use with caution)
$ git branch -D feature/login

# Rename current branch
$ git branch -m new-name

```

6.3 Branch Naming Conventions

Professional teams follow consistent naming conventions for branches. The most common approach:

Pattern	Purpose	Example
main / master	Production-ready code	main
develop	Integration branch for features	develop
feature/	New feature development	feature/user-auth
bugfix/	Bug fixes	bugfix/login-crash
hotfix/	Urgent production fixes	hotfix/sql-injection

Pattern	Purpose	Example
release/	Release preparation	release/v2.0.0
chore/	Maintenance tasks	chore/update-deps

6.4 How Branching Works Internally

When you make commits on a branch, the branch pointer advances forward with each commit. The other branches stay where they are:

```

STEP 1: Create and switch to feature branch

      main ■■■■■■■■■■ [C3]
feature ■■■■■■■■■■ [C3] ← both point to same commit
      HEAD ■■■■■■■■■■ feature

STEP 2: Make commit C4 on feature branch

      main ■■■■■■■■■■ [C3]
                    [C3] ■■■■ [C4] ← new commit
feature ■■■■■■■■■■ [C4]
      HEAD ■■■■■■■■■■ feature

STEP 3: Make commit C5 on feature branch

      main ■■■■■■■■■■ [C3]
                    [C3] ■■■■ [C4] ■■■■ [C5]
feature ■■■■■■■■■■ [C5]
      HEAD ■■■■■■■■■■ feature

main still points to C3. feature branch diverged from main.
```

6.5 Understanding Merge Conflicts

A **merge conflict** occurs when two branches have made different changes to the same region of the same file. Git cannot automatically decide which version to keep, so it asks you.

```

# When a conflict occurs during merge:
$ git merge feature/login
Auto-merging app.py
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.

# Look inside app.py – Git marks the conflict:
```

```

<<<<<<< HEAD                                     ← your current branch version
def login(email, password):
    user = db.find_by_email(email)
    if not user:
        return None
=====                                           ← separator
def login(username, password):
    user = db.find_by_username(username)
    if not user:
        raise UserNotFoundError(username)
>>>>>>> feature/login                             ← incoming branch version

# YOU must edit the file to resolve, then:
$ git add app.py
$ git commit -m 'merge: resolve login function signature conflict'

```

■ Tip

Conflicts are not errors — they are Git asking you to make a decision. Always read both versions carefully before choosing. Sometimes the correct resolution combines elements from both versions.

Section 7 — Merging and Rebasing

Once you have diverging branches, you need to integrate their changes. Git provides two fundamentally different strategies: **merging** and **rebasing**. Understanding the difference — both mechanically and philosophically — is essential for professional Git usage.

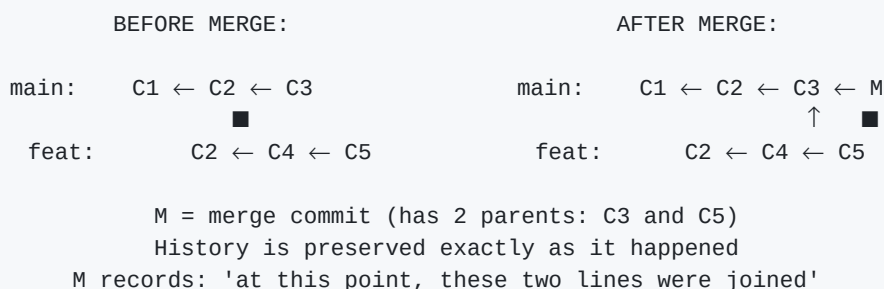
7.1 git merge

Merging integrates changes from one branch into another while **preserving the complete history** of both branches. It creates a new 'merge commit' that has two parents.

```
# Switch to the target branch (where you want changes to go)
$ git switch main

# Merge the feature branch into main
$ git merge feature/login

# If clean merge: Git creates a merge commit automatically
# If conflict: resolve, stage, commit manually
```



Fast-Forward Merge

A special case: if the target branch has not diverged from the source (i.e., the target is directly behind), Git simply moves the pointer forward. No merge commit is created.

```
BEFORE:                AFTER:

main: C1 ← C2           main: C1 ← C2 ← C3 ← C4
```

```

    ■
feat: C2 ← C3 ← C4                                (main pointer moved forward)

$ git merge feature → Fast-forward

# Force a merge commit even for fast-forward:
$ git merge --no-ff feature/login -m 'Merge feature/login into main'

# Teams often use --no-ff in GitHub PRs to preserve context

```

7.2 git rebase

Rebasing takes the commits from one branch and **replays them on top of another branch**. The result is a **linear history** — as if the work was done sequentially, not in parallel.

```

# Rebase feature branch onto main
$ git switch feature/login
$ git rebase main

# OR in one command:
$ git rebase main feature/login

```

BEFORE REBASE:	AFTER REBASE:
<pre> main: C1 ← C2 ← C3 ■ feat: C2 ← C4' ← C5' </pre>	<pre> main: C1 ← C2 ← C3 ■ feat: C3 ← C4'' ← C5'' </pre>
C4' and C5' are NEW commits (different SHA hashes!) They have the same changes but new parents History looks linear – as if feat was branched from C3 IMPORTANT: C4' ≠ C4 (different SHA, different parent) Rebasing REWRITES HISTORY	

7.3 Merge vs Rebase — The Real Difference

Aspect	git merge	git rebase
History	Preserves exact history, branches visible	Linear history, branches hidden

Aspect	git merge	git rebase
Merge commits	Creates merge commits	No merge commits
Safety	Safe to use on shared branches	NEVER rebase shared/public branches
Conflict resolution	Resolve once	Resolve per commit being replayed
Readability	Shows when and how features merged	Cleaner, sequential story
Use case	Merging features into main, PRs	Cleaning up local commits before PR

■ Danger

The Golden Rule of Rebasing: Never rebase commits that have been pushed to a shared remote branch. Rebasing creates new commits with new SHAs — if teammates have the old commits, their history will diverge and cause severe confusion. Rebase is safe only on your local, un-pushed commits.

7.4 Interactive Rebase — Powerful History Editing

Interactive rebase (**git rebase -i**) lets you rewrite your local commit history before pushing. You can reorder, edit, squash, or drop commits.

```
# Interactively rebase the last 4 commits
$ git rebase -i HEAD~4

# This opens your editor with:
pick a3f4b2c feat: add login page
pick 7e9c121 fix: typo in login page
pick 2d8f4ab feat: add logout button
pick 5b1e9c3 fix: logout redirect bug

# Commands you can use:
# pick    = use commit as-is
# reword  = use commit, but edit message
# edit    = use commit, but stop to amend
# squash  = combine with previous commit
# fixup   = like squash, discard this commit's message
# drop    = remove this commit entirely

# Example: squash typo fix into the login commit
```

```
pick a3f4b2c feat: add login page
squash 7e9c121 fix: typo in login page    ← becomes part of above
pick 2d8f4ab feat: add logout button
fixup 5b1e9c3 fix: logout redirect bug    ← merged, message discarded

# Result: clean 2-commit history instead of 4
```

7.5 Conflict Resolution Strategy

When conflicts occur (during either merge or rebase), follow this systematic approach:

- **1. Understand both versions:** Read HEAD version and incoming version carefully.
- **2. Understand the intent:** What was each developer trying to accomplish?
- **3. Write the correct resolution:** Often a combination of both changes.
- **4. Remove conflict markers:** Delete all <<<<<<, =====, >>>>>> lines.
- **5. Test your resolution:** The code must work after the merge.
- **6. Stage and continue:** git add, then git commit or git rebase --continue.

```
# After resolving conflicts in all files:
$ git add .                # Stage resolutions
$ git commit               # For merge conflicts
# OR
$ git rebase --continue    # For rebase conflicts

# To abort and go back to pre-conflict state:
$ git merge --abort
$ git rebase --abort

# Use a visual merge tool:
$ git mergetool
```

Section 8 — GitHub Explained

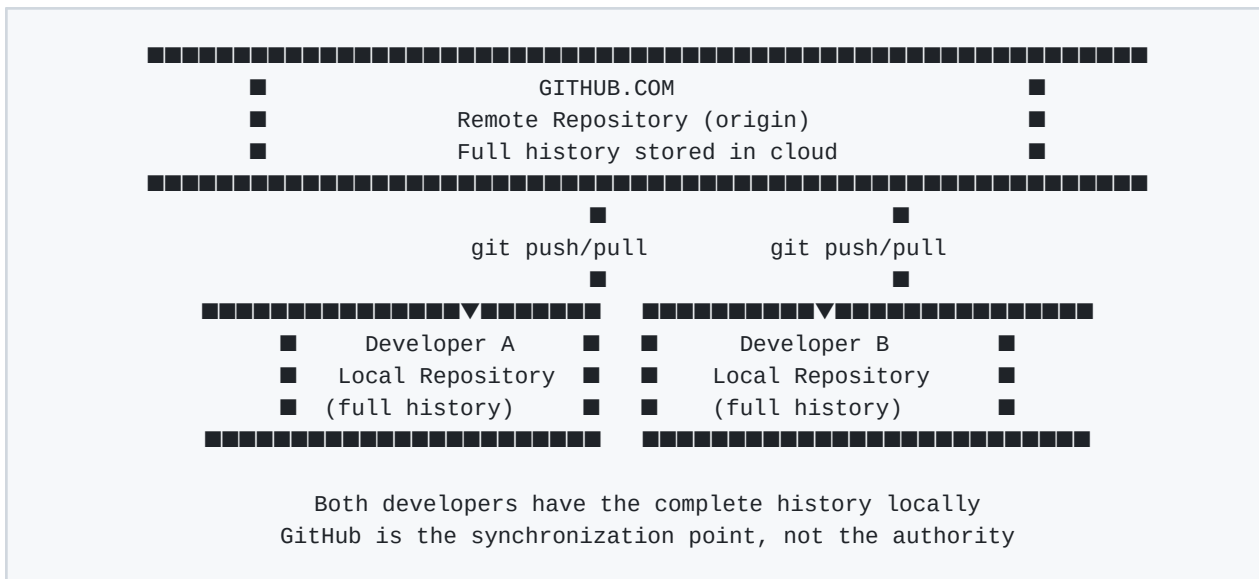
A common confusion among beginners: **Git and GitHub are not the same thing**. Git is the version control software that runs locally on your machine. GitHub is a web-based platform built on top of Git that adds collaboration features, a web UI, and cloud hosting.

8.1 Git vs GitHub

Aspect	Git	GitHub
What it is	Distributed VCS software	Cloud platform for Git repos
Created by	Linus Torvalds, 2005	Tom Preston-Werner et al., 2008
Where it runs	Your local machine	github.com servers
Cost	Free, open source	Free for public repos; paid plans
Purpose	Track changes locally	Collaboration, hosting, CI/CD
Alternatives	Mercurial, SVN	GitLab, Bitbucket, Gitea
Network required	No — works offline	Yes — cloud service

8.2 Remote Repositories

A **remote repository** is a version of your project hosted on a server (GitHub, GitLab, Bitbucket, or your own server). Remotes allow teams to share code and synchronize work.



8.3 Pull Requests (PRs)

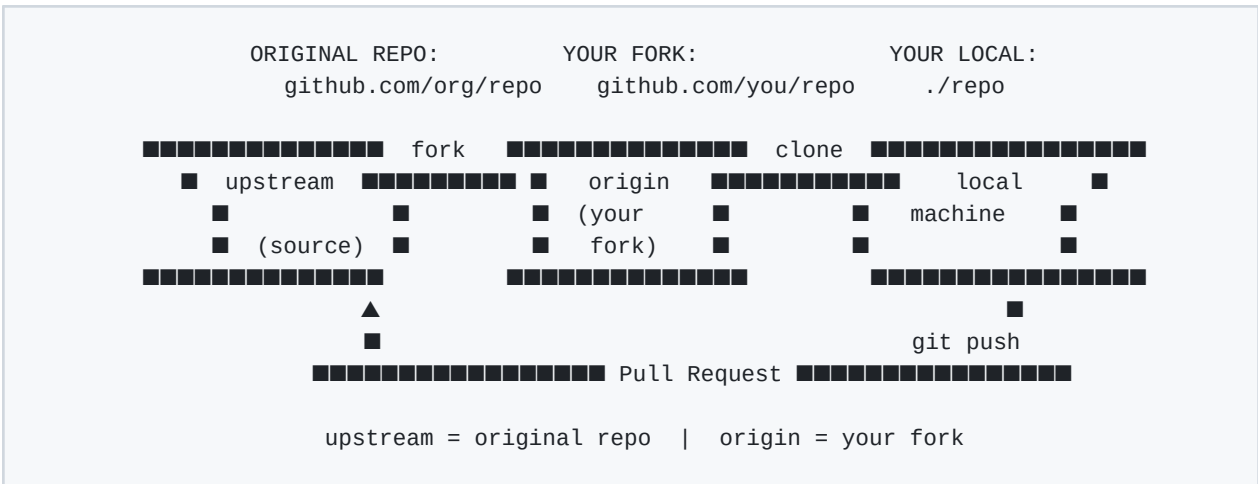
A **Pull Request** is a GitHub feature (not a Git feature) that lets you propose changes from one branch and request that they be merged into another. It is the central mechanism for code review in professional teams.

The Pull Request workflow:

- **1. Developer** pushes a feature branch to GitHub
- **2. Developer** opens a Pull Request from feature branch → main
- **3. Teammates** review the code, leave comments, request changes
- **4. Developer** makes requested changes, pushes more commits
- **5. Reviewers approve** the PR
- **6. Maintainer merges** the PR into main
- **7. Branch is deleted** (the feature branch is no longer needed)

8.4 Forks

A **fork** is a copy of a repository on GitHub's servers. Forking is used when you want to contribute to someone else's repository that you do not have write access to.



```
# Fork workflow:
# 1. Fork on GitHub UI
# 2. Clone your fork
$ git clone https://github.com/YOU/repo.git

# 3. Add upstream remote
$ git remote add upstream https://github.com/ORIGINAL/repo.git

# 4. Keep your fork updated:
$ git fetch upstream
$ git switch main
$ git merge upstream/main
$ git push origin main

# 5. Create feature branch, make changes, push to your fork
# 6. Open PR from your fork to upstream repo
```

8.5 GitHub Features Overview

Feature	Purpose
Issues	Bug tracking, feature requests, task management
Pull Requests	Code review and controlled merging
Actions	CI/CD automation workflows
Discussions	Community Q&A; and forums
Wiki	Project documentation

Feature	Purpose
Projects	Kanban-style project management
Releases	Versioned software releases with changelogs
Security Advisories	Responsible disclosure of vulnerabilities
Dependabot	Automatic dependency update PRs
Code Search	Search across all GitHub repositories

Section 9 — Connecting Local Repo to GitHub

Now we connect your local Git repository to GitHub. This involves configuring remotes, authenticating with GitHub, and understanding how data flows between local and remote.

9.1 Creating a GitHub Repository

- Go to **github.com** → click the + icon → **New repository**
- Choose a name, set visibility (public/private)
- Do NOT initialize with README if you have existing local code
- Click **Create repository**

9.2 Connecting Your Local Repo

```
# Add the GitHub URL as 'origin' remote
$ git remote add origin https://github.com/USERNAME/REPO.git

# Verify the remote was added:
$ git remote -v
origin https://github.com/USERNAME/REPO.git (fetch)
origin https://github.com/USERNAME/REPO.git (push)

# Push your local main branch to GitHub for the first time:
$ git push -u origin main

# The -u flag sets 'upstream' tracking
# After this, just 'git push' and 'git pull' work without arguments

# Change remote URL (e.g., switch from HTTPS to SSH):
$ git remote set-url origin git@github.com:USERNAME/REPO.git

# Remove a remote:
$ git remote remove origin
```

9.3 Authentication — HTTPS vs SSH

GitHub requires authentication to push code. There are two methods:

HTTPS Authentication

With HTTPS, you use a **Personal Access Token (PAT)** instead of your password (GitHub removed password authentication in 2021).

```
# Generate a PAT:
# GitHub → Settings → Developer Settings → Personal Access Tokens
# Select scopes: repo, workflow

# When Git prompts for password, enter your PAT instead
Username: your-github-username
Password: ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX ← PAT, not password

# Cache credentials to avoid repeated prompts:
$ git config --global credential.helper store      # Stores in plaintext (not ideal)
$ git config --global credential.helper cache      # Stores in memory temporarily

# On macOS: use the keychain
$ git config --global credential.helper osxkeychain

# On Windows: Git Credential Manager is included with Git for Windows
$ git config --global credential.helper manager
```

SSH Authentication (Recommended)

SSH key pairs are the professional standard. A **private key** stays on your machine. The **public key** is uploaded to GitHub. When you connect, GitHub verifies your identity using cryptographic challenge-response — no password ever transmitted.

```
# Step 1: Generate SSH key pair
$ ssh-keygen -t ed25519 -C "your@email.com"
# Press Enter to accept default location (~/.ssh/id_ed25519)
# Enter a strong passphrase (recommended)

# Step 2: Start SSH agent
$ eval "$(ssh-agent -s)"
```



```
# Then merge when ready:
$ git merge origin/main
```

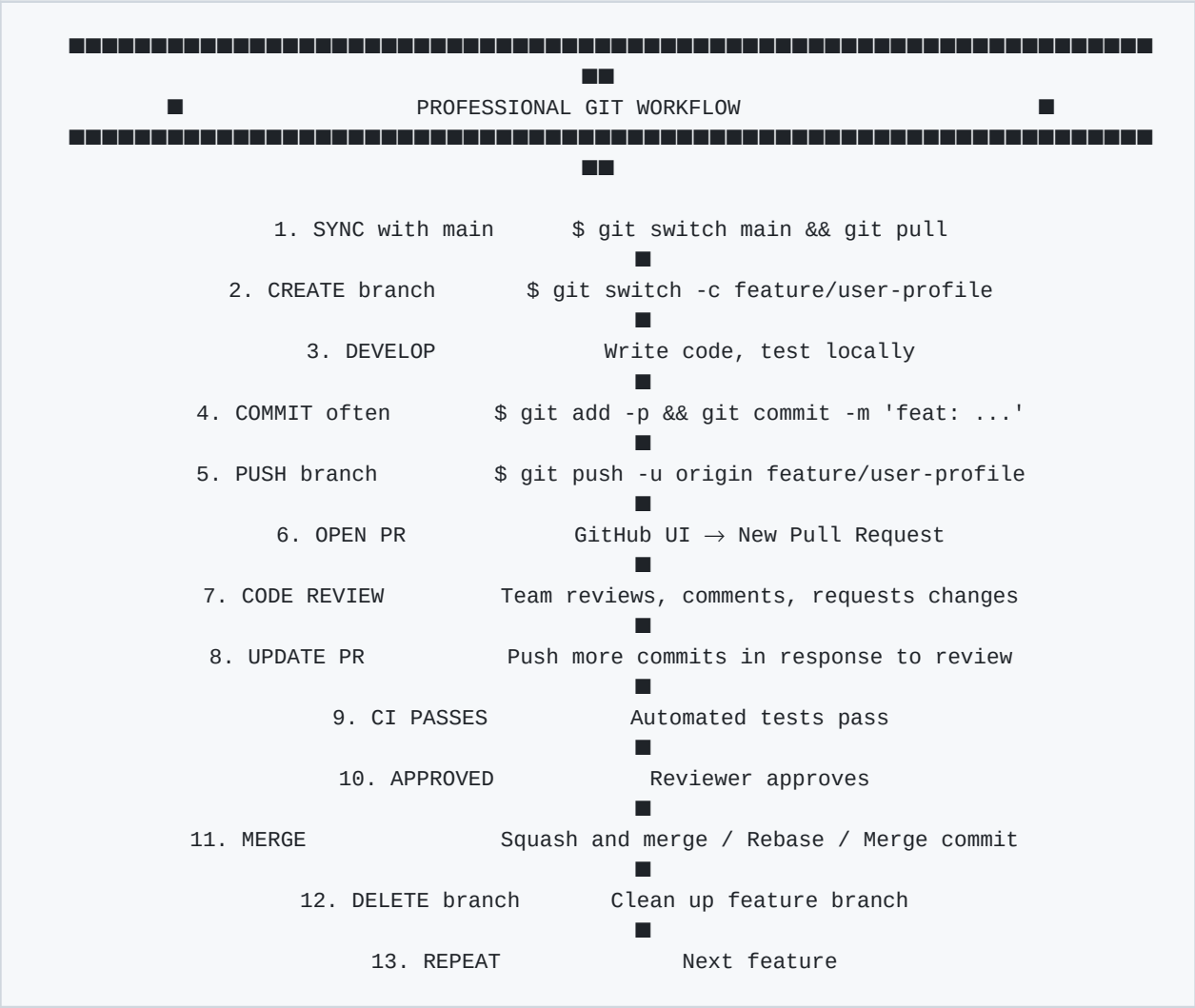
```
# Fetch + merge in one command
$ git pull origin main
```

```
# Set pull to always rebase:
$ git config --global pull.rebase true
```

Section 10 — Professional GitHub Workflow

In real teams, you never commit directly to main. Everything goes through branches and Pull Requests. This section walks you through the complete professional workflow used at companies ranging from startups to FAANG.

10.1 The Complete Feature Development Cycle



10.2 Step-by-Step with Commands

```
# ■■■ STEP 1: Always start from an updated main ████████████████████████████████
```

```
$ git switch main
```

```
$ git pull origin main
```

```
# ■■■ STEP 2: Create feature branch ████████████████████████████████████████████
```

```
$ git switch -c feature/user-profile
```

```
# ■■■ STEP 3: Write code, then commit in logical increments ██████████
```

```
# (Never one giant commit at the end)
```

```
$ git add src/components/UserProfile.jsx
```

```
$ git commit -m 'feat(profile): add UserProfile component'
```

```
$ git add src/api/users.py
```

```
$ git commit -m 'feat(api): add GET /users/:id endpoint'
```

```
$ git add tests/test_users.py
```

```
$ git commit -m 'test(api): add unit tests for user endpoint'
```

```
# ■■■ STEP 4: Push branch to GitHub ████████████████████████████████████████████
```

```
$ git push -u origin feature/user-profile
```

```
# ■■■ STEP 5: If main has moved forward, rebase before PR ████████████████████
```

```
$ git fetch origin
```

```
$ git rebase origin/main
```

```
$ git push --force-with-lease # Required after rebase
```

```
# ■■■ STEP 6: After PR is merged, clean up ████████████████████████████████████████████
```

```
$ git switch main
```

```
$ git pull origin main
```

```
$ git branch -d feature/user-profile
```

10.3 Writing a Professional Pull Request

A Pull Request description is documentation. Write it as if the reviewer has no context about what you were working on.

```
## Summary
Implement user profile page with avatar, bio, and edit functionality.

## Changes
```

```

- Add UserProfile React component with responsive layout
- Add GET /api/users/:id endpoint with proper error handling
- Add profile update endpoint with input validation
- Add unit tests for new API endpoints (93% coverage)

## Why These Changes
Part of the user onboarding flow redesign (Epic #23).
Users reported they could not find how to update their profiles.

## How to Test
1. Log in as any test user
2. Navigate to /profile
3. Verify profile loads correctly
4. Click 'Edit Profile' and submit changes

## Screenshots
[attach screenshots of the UI changes]

## Related Issues
Closes #45, #67

```

10.4 Repository Structure Best Practices

```

my-project/
├── README.md           ← Project overview, setup instructions
├── CONTRIBUTING.md     ← How to contribute
├── CHANGELOG.md        ← Version history
├── LICENSE             ← License file
├── .gitignore          ← What Git ignores
├── .github/
│   ├── workflows/     ← GitHub Actions CI/CD
│   ├── PULL_REQUEST_TEMPLATE.md
│   └── ISSUE_TEMPLATE/
├── src/                ← Source code
├── tests/              ← Test files
├── docs/               ← Documentation
└── scripts/           ← Utility scripts

```

10.5 README Best Practices

A great README includes:

- **Project title and description:** What it does in one sentence
- **Badges:** Build status, coverage, license
- **Screenshots/Demo:** Visual overview of the project
- **Features:** Key capabilities
- **Prerequisites:** What the user needs before installing
- **Installation:** Step-by-step setup commands
- **Usage:** How to use it with examples
- **Configuration:** Environment variables, config files
- **Testing:** How to run the test suite
- **Contributing:** Link to CONTRIBUTING.md
- **License:** License name and link
- **Contact:** How to reach the maintainer

Section 11 — Important Git Commands

This section covers the commands you will use regularly beyond the basics. Each command is explained with its internal mechanism, common use cases, and potential dangers.

11.1 git log — Exploring History

```
# Standard log
$ git log

# Compact one-line format
$ git log --oneline

# Beautiful graph view (add to alias!)
$ git log --oneline --graph --decorate --all

# Detailed stats (which files changed)
$ git log --stat

# Show patches (actual changes)
$ git log -p

# Custom format
$ git log --format='%h %an %ar: %s'
a3f4b2c Jane Smith 2 days ago: feat(auth): add JWT refresh

# Limit to N commits
$ git log -5

# Filter by author, date, or message
$ git log --author="Jane" --since="2024-01-01" --grep="auth"

# See who changed a specific file:
$ git log --follow -p src/auth/login.py
```

11.2 git diff — Comparing Changes

```
# Difference between working directory and staging area
$ git diff

# Difference between staging area and last commit
$ git diff --staged
# (same as --cached)

# Compare two commits
$ git diff a3f4b2c 7e9c121

# Compare two branches
$ git diff main feature/login

# See only changed filenames
$ git diff --name-only main feature/login

# See a summary of changes (files + insertions/deletions)
$ git diff --stat main feature/login

# Compare specific file between commits
$ git diff HEAD~3 HEAD -- src/app.py
```

11.3 git stash — Temporary Storage

git stash temporarily shelves changes you are not ready to commit. Perfect for when you need to switch branches quickly without committing half-finished work.

```
# Stash all uncommitted changes
$ git stash
Saved working directory and index state WIP on main: a3f4b2c feat(auth)

# Stash with a descriptive name
$ git stash push -m 'WIP: halfway through user profile'

# Stash including untracked files
$ git stash push --include-untracked

# List all stashes
$ git stash list
stash@{0}: WIP: halfway through user profile
```

```
# Apply most recent stash (keeps it in list)
$ git stash apply
```

```
# Apply a specific stash
$ git stash apply stash@{1}
```

```
# Apply AND remove from list
$ git stash pop
```

```
# Delete a stash
$ git stash drop stash@{0}
```

```
# Delete all stashes
$ git stash clear
```

git reset is powerful and can be destructive. It moves the current branch pointer backward.

[illegible]

```
# Moves HEAD back, keeps changes STAGED
# Use case: undo last commit but keep changes ready to re-commit
$ git reset --soft HEAD~1
```

[illegible]

```
# Moves HEAD back, unstages changes, keeps files on disk
# Use case: undo commit and unstage, inspect changes
$ git reset HEAD~1
$ git reset --mixed HEAD~1 # same thing
```

```
# git reset --hard
```

```
# Moves HEAD back, DISCARDS all changes permanently
# Use case: completely undo commits (DESTRUCTIVE)
$ git reset --hard HEAD~1
```

```
# Discard ALL local changes (back to last commit):
$ git reset --hard HEAD
```

```
# Reset specific file (unstage it):
```

```
$ git reset HEAD src/app.py
```

■ Danger

git reset --hard permanently discards changes in your working directory. These changes are NOT in the trash — they are gone. Double-check before using. Only use on local, un-pushed commits.

11.5 git revert — Safe Undo for Shared History

git revert creates a NEW commit that undoes the changes of a previous commit. Unlike reset, it does not rewrite history — safe to use on shared branches.

```
# Revert the last commit
$ git revert HEAD

# Revert a specific commit by hash
$ git revert a3f4b2c

# Revert without opening editor (use auto-generated message)
$ git revert --no-edit HEAD

# Revert multiple commits at once
$ git revert HEAD~3..HEAD

# After reverting, push normally:
$ git push origin main

# Because git revert adds commits, it does not rewrite history
```

11.6 git restore — Restoring Files

```
# Discard changes in working directory (restore to last commit)
$ git restore src/app.py

# Unstage a file (move from staging area back to working directory)
$ git restore --staged src/app.py

# Restore a file to a specific commit's version
$ git restore --source=HEAD~2 src/app.py

# Restore all files in directory
```

```
$ git restore .
```

11.7 git reflog — The Safety Net

git reflog is your safety net. It records every position HEAD has pointed to, including after resets, rebases, and branch switches. If you accidentally delete commits, reflog can help you recover them.

```
$ git reflog
a3f4b2c (HEAD -> main) HEAD@{0}: commit: feat(auth): add JWT refresh
7e9c121 HEAD@{1}: commit: fix(login): correct password hashing
2d8f4ab HEAD@{2}: checkout: moving from feature/login to main
5b1e9c3 HEAD@{3}: commit: feat(login): initial implementation

# Recover 'lost' commits after a bad reset:
$ git reset --hard HEAD~5    # Oops — went back too far!

# Find the commit you want in reflog
$ git reflog
# Look for the SHA before the reset

# Recover it:
$ git reset --hard HEAD@{2}
# OR create a branch at that point:
$ git branch recovery-branch HEAD@{2}
```

■ ■ Note

Reflog entries expire after 90 days by default. It is local only — reflog is NOT pushed to GitHub. It saves you from local mistakes, but not from remote history rewrites.

11.8 git cherry-pick

git cherry-pick applies a specific commit from one branch to your current branch. Useful for hotfixes or pulling individual features.

```
# Apply a specific commit to current branch
$ git cherry-pick a3f4b2c

# Cherry-pick a range of commits
```

```
$ git cherry-pick a3f4b2c..7e9c121

# Cherry-pick without committing (stage only)
$ git cherry-pick --no-commit a3f4b2c

# Real-world use case:
# A critical fix was committed to feature branch but you need it in main NOW
$ git switch main
$ git cherry-pick fix-commit-sha
```

11.9 git tag — Marking Releases

```
# Create lightweight tag (just a pointer)
$ git tag v1.0.0

# Create annotated tag (recommended — has metadata)
$ git tag -a v1.0.0 -m 'Release version 1.0.0'

# Tag a specific commit
$ git tag -a v0.9.0 a3f4b2c -m 'Beta release'

# List all tags
$ git tag

# Push tags to GitHub
$ git push origin --tags

# Delete a local tag
$ git tag -d v1.0.0

# Delete a remote tag
$ git push origin --delete v1.0.0
```

11.10 git blame — Finding Who Changed What

```
# Show who last modified each line of a file
$ git blame src/app.py

a3f4b2c (Jane Smith 2024-01-15 10:22:00 +0000 1) from flask import Flask
7e9c121 (Bob Johnson 2024-01-10 09:15:30 +0000 2) from auth import jwt
```

```
a3f4b2c (Jane Smith 2024-01-15 10:22:00 +0000 3)
5b1e9c3 (Alice Lee 2024-01-05 14:00:00 +0000 4) app = Flask(__name__)

# Use -L to blame only specific lines:
$ git blame -L 10,20 src/app.py

# Useful in VS Code: install GitLens extension for inline blame
```

11.11 git clean — Removing Untracked Files

```
# Preview what would be removed (DRY RUN — always do this first)
$ git clean -n
Would remove temp.txt
Would remove build/

# Remove untracked files
$ git clean -f

# Remove untracked files AND directories
$ git clean -fd

# Remove ignored files too (be very careful)
$ git clean -fdx
```

■ Danger

git clean -fd permanently deletes untracked files. These bypass the trash — they are gone immediately. ALWAYS run **git clean -n** first to preview what will be deleted.

Section 12 — Git Internals

This section goes deep. Understanding Git's internal object model will make every Git operation make sense at a fundamental level. You will stop thinking of Git as a black box and start understanding exactly what happens when you run any command.

12.1 The Object Database

Everything in Git is stored as an **object** in the **.git/objects** directory. There are exactly four types of objects. Together, they represent your entire repository history.

Object Type	Contains	Represents
blob	File content (raw bytes)	A single file at a point in time
tree	List of blob/tree pointers + names	A directory structure
commit	Tree pointer + parent + metadata	A snapshot with context
tag	Object pointer + tagger metadata	An annotated tag

The Blob Object

A **blob** (Binary Large Object) stores the raw content of a file. It does not store the filename or any metadata — only the content. Two files with identical content share one blob.

```
# Manually create a blob:
$ echo 'Hello, Git!' | git hash-object --stdin -w
8ab686eafeb1f44702738c8b0f24f2567c36da6d

# The blob is now stored:
$ file .git/objects/8a/b686eafeb1f44702738c8b0f24f2567c36da6d
.git/objects/8a/b686...: zlib compressed data

# Read the blob content:
$ git cat-file -p 8ab686
Hello, Git!

# Format of a blob before hashing:
# 'blob <size>\0<content>'
```

```
# SHA1('blob 12\0Hello, Git!\n') = 8ab686...
```

The Tree Object

A **tree** object represents a directory. It contains a list of entries, each with: permissions, object type, SHA hash, and name.

```
# Inspect a tree object:
$ git cat-file -p HEAD^{tree}

100644 blob a3f4b2c src/app.py
100644 blob 7e9c121 src/models.py
040000 tree 2d8f4ab src/templates/
100644 blob 5b1e9c3 README.md
100644 blob 9d2c8f1 requirements.txt

# File permissions:
# 100644 = regular file
# 100755 = executable file
# 120000 = symbolic link
# 040000 = directory (tree)
```

The Commit Object

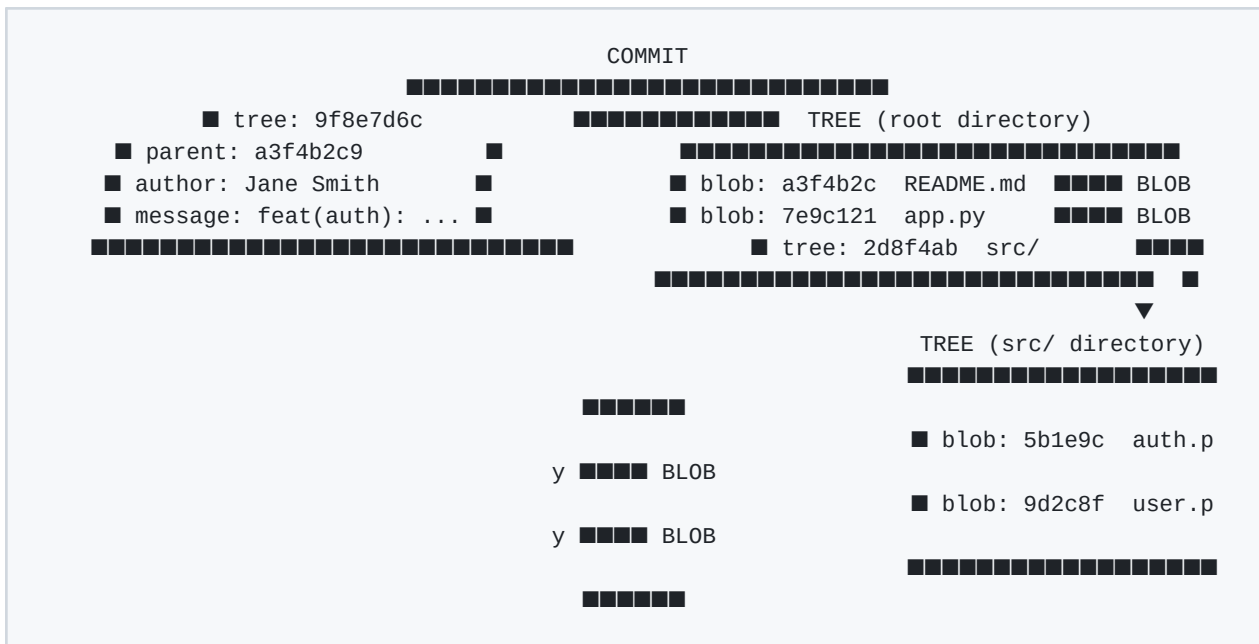
A **commit** object ties everything together. It points to a tree (the snapshot), the parent commit(s), and records authorship and message.

```
$ git cat-file -p HEAD

tree 9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e
parent a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5
author Jane Smith <jane@example.com> 1705312800 +0000
committer Jane Smith <jane@example.com> 1705312800 +0000
pgpsig -----BEGIN PGP SIGNATURE-----      ← optional signed commit
...
-----END PGP SIGNATURE-----

feat(auth): add JWT token refresh endpoint
```

12.2 The Complete Object Graph



12.3 References and Refs

References (refs) are human-readable names pointing to SHA hashes. They live in **.git/refs/**.

```

.git/refs/
  ■■■ heads/          ← local branch pointers
  ■   ■■■ main        ← content: SHA of latest main commit
  ■   ■■■ feature/login ← content: SHA of latest feature commit
  ■■■ tags/          ← tag pointers
  ■   ■■■ v1.0.0       ← content: SHA of tagged commit
  ■■■ remotes/       ← remote tracking refs
    ■■■ origin/
      ■■■ main         ← where origin/main was last seen
      ■■■ feature/login

# Packed refs (stored in one file for efficiency):
$ cat .git/packed-refs
# pack-refs with: peeled fully-peeled sorted
a3f4b2c9... refs/heads/main
7e9c1210... refs/tags/v1.0.0
9f8e7d6c... refs/tags/v1.0.0^{*} ← dereferenced to commit

```

12.4 HEAD and Detached HEAD

HEAD is a special reference file that points to the currently checked-out branch or commit.

```
# Normal state (HEAD points to branch):
$ cat .git/HEAD
ref: refs/heads/main

# DETACHED HEAD state (HEAD points directly to commit):
$ git checkout a3f4b2c
HEAD is now at a3f4b2c feat(auth): add JWT refresh

$ cat .git/HEAD
a3f4b2c9d8e1f0a7b6c5d4e3f2a1b0c9d8e7f6a5

# In detached HEAD: commits you make are not on any branch
# They will be garbage collected eventually!

# To save work done in detached HEAD:
$ git switch -c new-branch-name
# This creates a branch at the current commit
```

NORMAL:



DETACHED HEAD:



12.5 Object Packing

Initially, Git stores each object as a separate compressed file (loose objects). As the repository grows, Git periodically runs **git gc** (garbage collection) which packs loose objects into **packfiles** — highly efficient binary files with delta compression.

```
# Run garbage collection manually:
$ git gc

# Check repository disk usage:
$ git count-objects -vH
count: 0          ← loose objects
size: 0 bytes
in-pack: 1842     ← packed objects
```

```
packs: 1
size-pack: 3.87 MiB

# Packfiles live in:
.git/objects/pack/
■■■ pack-a3f4b2c...sha1.pack    ← the data
■■■ pack-a3f4b2c...sha1.idx    ← the index for fast lookup
```

Section 13 — Common Beginner Mistakes

Every developer makes these mistakes when learning Git. Understanding why they are problematic helps you build the right habits from the start.

13.1 Working Directly on main

Why it is dangerous: Direct commits to main bypass the PR review process, risk breaking production, and make collaboration difficult. If something breaks, there is no clean way to isolate the problem.

How to fix it: Always create a feature branch. main should only receive changes through reviewed PRs.

```
# ■ Wrong:
$ git switch main
$ # ...make changes...
$ git commit -m 'add feature'
$ git push origin main

# ■ Correct:
$ git switch -c feature/my-feature
$ # ...make changes...
$ git commit -m 'feat: add feature'
$ git push origin feature/my-feature
# → Open Pull Request on GitHub
```

13.2 Giant Commits

Why it is dangerous: Committing hundreds of changes in one commit makes code review impossible, makes bisecting bugs impractical, and provides no useful history. If you need to revert one feature, you revert all of them.

How to fix it: Commit in small, logical units. Each commit should represent one coherent change.

```
# ■ Wrong: one massive commit after days of work
git commit -m 'add all the features and fix bugs and update deps'

# ■ Correct: logical progression
git commit -m 'feat(user): add user registration model'
git commit -m 'feat(api): add POST /register endpoint'
git commit -m 'test(api): add registration endpoint tests'
git commit -m 'feat(email): add registration confirmation email'
```

13.3 Pushing Secrets to GitHub

Why it is dangerous: API keys, database passwords, JWT secrets, and credentials pushed to GitHub are COMPROMISED immediately. Bots scan GitHub for secrets 24/7. Even private repositories have been compromised. Even deleting the file does not help — the secret is still in Git history.

How to fix it: Use .gitignore for .env files. Use environment variables. Use secret managers (HashiCorp Vault, AWS Secrets Manager). If you accidentally push: rotate the credential IMMEDIATELY, then clean history.

```
# ■ NEVER do this:
# app.py
DATABASE_URL = 'postgresql://admin:password123@prod.db.com/app'
STRIPE_SECRET_KEY = 'sk_live_xxxxxxxxxxxxx'

# ■ Use environment variables:
import os
DATABASE_URL = os.getenv('DATABASE_URL')
STRIPE_SECRET_KEY = os.getenv('STRIPE_SECRET_KEY')

# .env file (NOT committed to Git):
DATABASE_URL=postgresql://admin:password123@prod.db.com/app
STRIPE_SECRET_KEY=sk_live_xxxxxxxxxxxxx

# .gitignore (COMMITTED):
.env
.env.*
```

13.4 Force Pushing to Shared Branches

Why it is dangerous: `git push --force` overwrites the remote branch history. If teammates have pulled those commits, their history will diverge and they will face unexplainable errors. This can destroy collaborative work.

How to fix it: Never force push to main or shared branches. Use `--force-with-lease` (safer) only on your own feature branches after rebasing. Never force push to main.

```
# ■ Dangerous:
$ git push --force origin main

# ■ Safer (fails if remote has unexpected commits):
$ git push --force-with-lease origin feature/my-branch
# force-with-lease = force push only if remote matches your expectation

# Never do this on shared branches!
```

13.5 Not Pulling Before Pushing

Why it is dangerous: If you have not fetched recent commits from the remote, your push will be rejected (non-fast-forward). Developers sometimes force push to solve this, which overwrites teammates' work.

How to fix it: Always fetch and integrate remote changes before pushing.

```
# ■ Wrong approach:
$ git push --force origin main # Overwrites teammates' work!

# ■ Correct approach:
$ git fetch origin
$ git rebase origin/main
$ git push origin main # Clean, no force needed
```

13.6 Forgetting .gitignore

Why it is dangerous: Without `.gitignore`, you risk accidentally committing `node_modules` (100MB+), compiled binaries, OS metadata files (`.DS_Store`), IDE settings, and logs. This bloats the repository and causes confusion.

How to fix it: Create `.gitignore` before the first commit. Use gitignore.io to generate templates.

```
# Generate a .gitignore template:
```

```
# https://www.toptal.com/developers/gitignore
# Enter: node, python, macos, windows, vscode

# If you already committed node_modules by mistake:
$ git rm -r --cached node_modules
$ echo 'node_modules/' >> .gitignore
$ git commit -m 'chore: remove node_modules from tracking'
```

Section 14 — Security Best Practices

Security in Git is often overlooked until something goes wrong. This section covers the practices that protect you, your team, and your users.

14.1 The .env File Strategy

Every project has secrets: API keys, database connection strings, OAuth credentials, encryption keys. The universal pattern is the **.env** file — a local file that contains environment-specific secrets and is NEVER committed to Git.

```
# Project structure:
my-project/
  ■■■ .env           ← NEVER commit (add to .gitignore)
  ■■■ .env.example   ← COMMIT this (template with fake values)
  ■■■ .gitignore     ← includes .env
  ■■■ src/

# .env.example (safe to commit — shows what variables are needed):
DATABASE_URL=postgresql://localhost/myapp
SECRET_KEY=your-secret-key-here
STRIPE_KEY=sk_test_your_key_here
AWS_ACCESS_KEY=your-aws-key-here

# .gitignore must include:
.env
.env.local
.env.*.local
.env.production
.env.staging
```

14.2 What to Do if Secrets Are Exposed

If you accidentally pushed secrets to GitHub, you must treat the credential as **compromised regardless of how quickly you delete the file or commit**. GitHub indexes history immediately. Bots scan for secrets in real-time.

- **Step 1:** Revoke and rotate the credential IMMEDIATELY — this is the priority.
- **Step 2:** Remove the secret from Git history using git-filter-repo.
- **Step 3:** Force push the cleaned history.
- **Step 4:** Notify teammates to re-clone the repository.
- **Step 5:** Add the secret pattern to .gitignore.

```
# Install git-filter-repo:
$ pip install git-filter-repo

# Remove a file from all history:
$ git filter-repo --path .env --invert-paths

# Remove a specific string from all history:
$ git filter-repo --replace-text <(echo 'sk_live_XXXX==>***REMOVED***')

# Force push cleaned history:
$ git push origin --force --all
$ git push origin --force --tags

# NOTE: Even after this, GitHub may cache the old content
# Contact GitHub support to purge caches
```

■ Danger

If the secret was exposed even for minutes on a public repository, assume it has been found. Credential rotation is mandatory — cleaning the history is secondary damage control.

14.3 SSH Key Security

- **Use Ed25519:** More secure and faster than RSA.
- **Use a passphrase:** Protects your private key if your machine is compromised.
- **One key per machine:** Do not copy private keys between machines.
- **Never share private keys:** Your private key should never leave your machine.
- **Review authorized keys:** Periodically audit SSH keys added to GitHub.

```
# Generate a strong key:
$ ssh-keygen -t ed25519 -C 'work-laptop-2024' -f ~/.ssh/id_ed25519_github

# Use SSH config for multiple GitHub accounts:
# ~/.ssh/config
```

```
Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work

Host github-personal
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal

# Then use aliases in remote URLs:
$ git remote add origin git@github-work:company/repo.git
```

14.4 Signed Commits

Commit signatures cryptographically verify that a commit was made by the claimed author. Without signing, anyone with repository access can impersonate you in a commit. GitHub shows a **Verified** badge on signed commits.

```
# Set up GPG signing:
$ gpg --gen-key
$ gpg --list-secret-keys --keyid-format=long

# Output:
sec  ed25519/3AA5C34371567BD2 2024-01-15 [SC]
...

# Configure Git to sign with your key:
$ git config --global user.signingkey 3AA5C34371567BD2
$ git config --global commit.gpgsign true

# Export public key to add to GitHub:
$ gpg --armor --export 3AA5C34371567BD2
# Copy output → GitHub → Settings → SSH and GPG Keys

# Verify a commit's signature:
$ git verify-commit HEAD
gpg: Good signature from 'Jane Smith <jane@example.com>'
```

14.5 Repository Security Settings (GitHub)

- **Protected branches:** Prevent direct pushes to main, require PRs and reviews
- **Required status checks:** Tests must pass before merging
- **Require signed commits:** Only accept verified commits on protected branches
- **CODEOWNERS:** Require specific reviewers for specific files
- **Dependabot alerts:** Automatic notifications for vulnerable dependencies
- **Secret scanning:** GitHub automatically scans for exposed credentials

```
# CODEOWNERS file (place in .github/ or root):  
# Pattern          Owners  
*.py               @backend-team  
src/auth/          @security-team  
*.tf               @devops-team  
docs/              @documentation-team  
  
# Required reviews: when auth/ changes, @security-team must approve the PR
```

Section 15 — Real Project Example

This section walks through a complete, realistic development scenario — building a **Realtime Chat Application** — using professional Git workflow from start to finish.

15.1 Project Setup

```
# Create the project
$ mkdir realtime-chat && cd realtime-chat
$ git init

# Create .gitignore before anything else
$ cat > .gitignore << 'EOF'
.env
.env.*
node_modules/
__pycache__/
venv/
*.pyc
dist/
.DS_Store
*.log
EOF

# Create README
$ cat > README.md << 'EOF'
# Realtime Chat Application

A WebSocket-based realtime chat application.

## Tech Stack
- Backend: Python (FastAPI + WebSockets)
- Frontend: Vanilla JS
- Database: PostgreSQL
EOF

# Initial commit
$ git add .
$ git commit -m 'chore: initialize project with .gitignore and README'
```

```
# Connect to GitHub
$ git remote add origin git@github.com:username/realtime-chat.git
$ git push -u origin main
```

15.2 Feature Branch: User Authentication

```
# Create feature branch from main
$ git switch main && git pull
$ git switch -c feature/user-authentication

# --- Developer writes code ---
# src/auth/models.py, src/auth/router.py, tests/test_auth.py

# Commit in logical units:
$ git add src/auth/models.py
$ git commit -m 'feat(auth): add User model with password hashing'

$ git add src/auth/router.py
$ git commit -m 'feat(auth): add register and login endpoints'

$ git add tests/test_auth.py
$ git commit -m 'test(auth): add unit tests for auth endpoints'

$ git add src/auth/middleware.py
$ git commit -m 'feat(auth): add JWT validation middleware'

# Push branch
$ git push -u origin feature/user-authentication

# → Open PR on GitHub
# PR title: 'feat(auth): implement user authentication with JWT'
```

15.3 Code Review Process

```
# Reviewer leaves comment:
# 'The JWT secret should be loaded from environment, not hardcoded'

# Developer fixes the issue:
# src/auth/router.py – remove hardcoded secret
```

```
$ git add src/auth/router.py
$ git commit -m 'fix(auth): load JWT secret from environment variable'
$ git push

# Reviewer approves
# PR is merged via 'Squash and merge'

# Clean up:
$ git switch main && git pull
$ git branch -d feature/user-authentication
```

15.4 Feature Branch: Chat Rooms

```
# Meanwhile, another developer works on chat rooms
$ git switch -c feature/chat-rooms

$ git commit -m 'feat(chat): add Room model and database schema'
$ git commit -m 'feat(chat): add WebSocket handler for chat rooms'
$ git commit -m 'feat(chat): add room creation and joining API'
$ git commit -m 'test(chat): add WebSocket connection tests'

# Before opening PR, sync with updated main:
$ git fetch origin
$ git rebase origin/main

# If conflicts: resolve, git add, git rebase --continue

# Push (force-with-lease because we rebased):
$ git push --force-with-lease origin feature/chat-rooms
```

15.5 Hotfix — Production Bug

```
# Production reports: messages not being saved to database

# IMMEDIATELY branch from main (NOT from feature branch):
$ git switch main && git pull
$ git switch -c hotfix/messages-not-persisting

# Fix the bug:
```

```
$ git add src/chat/handler.py
$ git commit -m 'fix(chat): await database transaction before closing WebSoc
t'

# Push and open URGENT PR:
$ git push -u origin hotfix/messages-not-persisting

# After fast PR review and merge to main:
$ git switch main && git pull

# Also tag the hotfix:
$ git tag -a v1.0.1 -m 'Hotfix: fix message persistence'
$ git push origin --tags

# Clean up:
$ git branch -d hotfix/messages-not-persisting
```

15.6 Release Workflow

```
# Prepare release v1.1.0
$ git switch main && git pull
$ git switch -c release/v1.1.0

# Final testing, update version number, update CHANGELOG
$ git commit -m 'chore(release): bump version to 1.1.0'
$ git commit -m 'docs: update CHANGELOG for v1.1.0'

# Merge to main
$ git switch main
$ git merge --no-ff release/v1.1.0 -m 'Release v1.1.0'

# Tag the release
$ git tag -a v1.1.0 -m 'Version 1.1.0 - Chat rooms feature'
$ git push origin main --tags

# Create GitHub Release for changelog visibility
# GitHub → Releases → Create new release → Select tag v1.1.0
```

Section 16 — GitHub Portfolio Tips

Your GitHub profile is your public resume as a developer. Recruiters and hiring managers actively review GitHub profiles. Here is how to make yours stand out.

16.1 What Recruiters Actually Look For

Signal	What It Demonstrates
Consistent activity (green squares)	Discipline, continuous learning
Clean commit history	Professional workflow, attention to detail
Thoughtful commit messages	Communication skills, engineering mindset
Well-documented READMEs	Ability to explain work, documentation habit
Diverse project types	Breadth of skills
Tests in projects	Quality mindset, senior engineering practices
CI/CD setup (.github/workflows)	DevOps awareness
Issue and PR usage	Team collaboration experience
Code review patterns (PRs)	Review skills, structured development

16.2 Profile README

GitHub allows a special repository named exactly like your username. The README in that repository is displayed on your profile page. Make it count.

```
# Create: github.com/YOUR-USERNAME/YOUR-USERNAME

# Example profile README structure:

# ■ Hi, I'm Jane Smith
```

```
# Senior Full-Stack Developer specializing in distributed systems
# and real-time applications.

# ## ■ Tech Stack
# Python · FastAPI · PostgreSQL · React · TypeScript · Docker · AWS

# ## ■ Featured Projects
# - **[Realtime Chat](github.com/jane/chat)** – WebSocket chat with 1k+ stars
# - **[ML Pipeline](github.com/jane/ml-pipeline)** – Automated ML training pipeline

# ## ■ GitHub Stats
# ![GitHub stats](https://github-readme-stats.vercel.app/api?username=jane)

# ## ■ Contact
# [LinkedIn](linkedin.com/in/jane) · [Blog](dev.to/jane) · jane@email.com
```

16.3 Project README Excellence

- **First impression:** What does it do? One clear sentence.
- **Demo:** Screenshot or GIF at the top — show, don't just tell.
- **Badges:** Build status, coverage, license, version.
- **Quick start:** User can run it in under 5 minutes.
- **Architecture:** Brief overview of how it's built.
- **Contributing:** How to contribute, linked to CONTRIBUTING.md.
- **Live demo link:** Deploy to Render, Railway, or Vercel.

16.4 Pinned Repositories

Pin your 6 best repositories. Choose projects that show:

- **Breadth:** Backend, frontend, tool, API, open-source contribution
- **Complexity:** At least 1-2 projects with real architecture decisions
- **Completeness:** Projects with tests, documentation, and deployment
- **Recency:** Active commits signal you are still growing

16.5 Commit History Best Practices

```
# Make atomic, meaningful commits
git commit -m 'feat(api): add rate limiting to auth endpoints'

# Not vague dump-commits
git commit -m 'work'

# Daily commits signal consistency to recruiters
# Even documentation, refactoring, or test additions count

# Use the contribution graph to your advantage:
# Commit to different repos across different days
# Work on open source contributions
# Personal projects, blog posts as repos, etc.
```

16.6 Open Source Contributions

Contributing to open source projects is one of the strongest portfolio signals. It shows you can work within an existing codebase, follow external contribution guidelines, and collaborate with unknown developers.

- **Start small:** Fix typos in documentation, improve error messages
- **Look for 'good first issue' labels** on popular repositories
- **Read CONTRIBUTING.md** before submitting any PR
- **Be patient:** Review can take days or weeks on busy projects
- **Respond to feedback professionally:** This is being evaluated too

Section 17 — Advanced Topics

This section covers advanced Git and GitHub topics that senior developers use regularly. These concepts move you from Git proficiency to Git mastery.

17.1 Git Hooks

Git hooks are scripts that run automatically when specific Git events occur. They live in `.git/hooks/` and are not tracked by Git (local only) unless you use a tool like Husky to manage them.

Hook	When It Runs	Common Use
pre-commit	Before commit is created	Lint, format, run quick tests
commit-msg	After commit message is written	Validate commit message format
pre-push	Before pushing to remote	Run full test suite
post-commit	After commit is created	Notifications, logging
pre-rebase	Before rebase begins	Safety checks
post-merge	After merge completes	Install dependencies
post-checkout	After branch switch	Update environment

```
# Example pre-commit hook that runs linting:
# .git/hooks/pre-commit (make executable: chmod +x)

#!/bin/bash
echo 'Running pre-commit checks...'

# Run Python linting
if ! ruff check src/; then
    echo 'Linting failed. Commit aborted.'
    exit 1
fi

# Run type checking
```

```

if ! mpy src/; then
    echo 'Type check failed. Commit aborted.'
    exit 1
fi

echo 'All checks passed!'
exit 0

# Using Husky (Node.js projects) for shared hooks:
$ npm install husky --save-dev
$ npx husky init

# .husky/pre-commit:
npm run lint && npm run typecheck

```

17.2 CI/CD with GitHub Actions

GitHub Actions is a CI/CD platform built into GitHub. When you push code, Actions automatically runs your tests, builds your project, deploys to staging, and more.

```

# .github/workflows/ci.yml

name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: testpassword
        ports:
          - 5432:5432

    steps:

```

```

- name: Checkout code
  uses: actions/checkout@v4

- name: Set up Python
  uses: actions/setup-python@v4
  with:
    python-version: '3.12'

- name: Install dependencies
  run: |
    pip install -r requirements.txt
    pip install -r requirements-dev.txt

- name: Run linting
  run: ruff check src/

- name: Run type checking
  run: mypy src/

- name: Run tests
  run: pytest tests/ --cov=src --cov-report=xml
  env:
    DATABASE_URL: postgresql://postgres:testpassword@localhost/testdb

- name: Upload coverage
  uses: codecov/codecov-action@v3

```

17.3 Protected Branches

Protected branches enforce team workflow rules and prevent accidents. Configure in **GitHub** → **Repository** → **Settings** → **Branches** → **Add branch protection rule**.

- **Require pull request before merging:** No direct pushes to main
- **Require approvals:** Minimum 1-2 reviewer approvals
- **Require status checks:** CI must pass before merge
- **Require signed commits:** Only GPG-signed commits accepted
- **Require linear history:** Enforce squash or rebase — no merge commits
- **Include administrators:** Even repository owners follow the rules
- **Restrict who can push:** Only specific teams or users

17.4 Semantic Versioning

Semantic Versioning (SemVer) is the standard for version numbers: **MAJOR.MINOR.PATCH**.

Version Part	When to Increment	Example
MAJOR (X.0.0)	Breaking changes — incompatible API changes	1.0.0 → 2.0.0
MINOR (0.X.0)	New backward-compatible features	1.2.0 → 1.3.0
PATCH (0.0.X)	Backward-compatible bug fixes	1.2.3 → 1.2.4

```
# Conventional commits map to SemVer automatically:

# PATCH bump (bug fix):
fix(auth): correct token expiry calculation

# MINOR bump (new feature):
feat(profile): add avatar upload support

# MAJOR bump (breaking change — use ! or BREAKING CHANGE footer):
feat(api)!: change authentication to OAuth2

# OR with footer:
refactor(api): restructure endpoints

BREAKING CHANGE: /api/v1/users is now /api/v2/users

# Tools like 'semantic-release' read commits and auto-bump versions
```

17.5 Monorepos

A **monorepo** is a single repository containing multiple projects. Used by Google, Facebook, Twitter, and many others. Advantages include unified versioning, easy cross-project changes, and shared tooling.

```
# Monorepo structure example:
company-platform/
├── packages/
│   ├── api/           ← Backend service
│   ├── web/           ← Frontend app
│   ├── mobile/        ← Mobile app
│   └── shared/         ← Shared utilities
```

```
■■■ tools/          ← Build tools
■■■ .github/workflows/ ← CI/CD
■■■ package.json    ← Workspace root
```

```
# Tools for monorepo management:
# - Turborepo (Next.js ecosystem)
# - Nx (enterprise)
# - Lerna (legacy, still used)
# - Bazel (Google's build system)

# Targeted CI – only test what changed:
$ turbo run test --filter='./packages/api'
```

17.6 Git Bisect — Finding Regression Bugs

git bisect uses binary search to find which commit introduced a bug. Extremely powerful for large projects.

```
# Start bisect session
$ git bisect start

# Tell Git: current commit is bad (bug exists)
$ git bisect bad HEAD

# Tell Git: a known good commit (before the bug)
$ git bisect good v1.0.0

# Git checks out a commit halfway between good and bad
# Test if the bug exists:
# If YES: $ git bisect bad
# If NO:  $ git bisect good

# Git keeps halving until it finds the exact bad commit
# For 1000 commits, finds in ~10 steps

# Automate with a test script:
$ git bisect run python test_feature.py
# Git runs the script on each checked-out commit
# Exit 0 = good, Exit 1 = bad

# End session:
$ git bisect reset
```

Section 18 — Professional Cheat Sheet

A complete reference of the most important Git commands, organized by category with danger levels.

18.1 Setup & Configuration

Command	Purpose	Danger
git config --global user.name 'Name'	Set your name globally	Safe
git config --global user.email 'e@mail'	Set your email globally	Safe
git config --list	View all configuration	Safe
git init	Initialize new repository	Safe
git clone	Clone remote repository	Safe

18.2 Daily Workflow

Command	Purpose	Danger
git status	Show working tree status	Safe
git add	Stage specific file	Safe
git add .	Stage all changes	Safe
git add -p	Stage changes interactively	Safe
git commit -m 'msg'	Commit staged changes	Safe
git commit --amend	Modify last commit	■■■ Rewrites
git push origin	Push branch to remote	Safe
git pull	Fetch and merge	Safe

Command	Purpose	Danger
git fetch	Download without merging	Safe

18.3 Branching

Command	Purpose	Danger
git branch	List local branches	Safe
git branch -a	List all branches (incl. remote)	Safe
git switch -c	Create and switch to branch	Safe
git switch	Switch branches	Safe
git branch -d	Delete branch (safe)	Safe
git branch -D	Force delete branch	■■■ Data loss
git branch -m	Rename current branch	Safe

18.4 Merging & Rebasing

Command	Purpose	Danger
git merge	Merge branch into current	Safe
git merge --no-ff	Merge with commit	Safe
git rebase main	Rebase onto main	■■■ Rewrites
git rebase -i HEAD~N	Interactive rebase (N commits)	■■■ Rewrites
git merge --abort	Abort in-progress merge	Safe
git rebase --abort	Abort in-progress rebase	Safe
git cherry-pick	Apply specific commit	Safe

18.5 Undoing Changes

Command	Purpose	Danger
git restore	Discard working dir changes	■■ Irreversible
git restore --staged	Unstage a file	Safe
git revert HEAD	Create undo commit (safe)	Safe
git reset --soft HEAD~1	Undo commit, keep staged	■■ Rewrites
git reset --mixed HEAD~1	Undo commit, unstage	■■ Rewrites
git reset --hard HEAD~1	Undo commit, discard changes	■ DESTRUCTIVE
git clean -fd	Remove untracked files	■ DESTRUCTIVE

18.6 Inspection & Debugging

Command	Purpose	Danger
git log --oneline --graph --all	Visual commit history	Safe
git diff	Working dir vs staged	Safe
git diff --staged	Staged vs last commit	Safe
git show	Show commit details	Safe
git blame	Show who changed each line	Safe
git reflog	History of HEAD movements	Safe
git bisect start/good/bad	Binary search for bugs	Safe
git stash / git stash pop	Temporarily save changes	Safe

18.7 Remote Operations

Command	Purpose	Danger
git remote -v	List remotes	Safe
git remote add origin	Add remote	Safe
git remote set-url origin	Change remote URL	Safe

Command	Purpose	Danger
git push -u origin	Push with tracking	Safe
git push --force-with-lease	Safe force push	■■ Use carefully
git push --force	Force push (overwrite remote)	■ DESTRUCTIVE
git fetch --prune	Fetch + remove stale refs	Safe

18.8 Danger Level Legend

Level	Meaning
Safe	No data can be lost. Reversible or read-only.
■■ Rewrites	Rewrites history. Safe on local/unshared branches only.
■■ Irreversible	Cannot easily be undone, but data is not deleted.
■ DESTRUCTIVE	Permanently deletes data. Think twice. Use reflog to check recovery options first.

18.9 Essential Git Aliases

```
# Add these to ~/.gitconfig under [alias]:

[alias]
  st = status
  co = checkout
  br = branch
  ci = commit
  lg = log --oneline --graph --decorate --all
  undo = reset --soft HEAD~1
  unstage = restore --staged
  aliases = config --get-regexp alias
  pushf = push --force-with-lease
  recent = branch --sort=-committerdate
  wip = commit -am 'WIP: work in progress [skip ci]'

# Now you can use:
$ git lg          # Beautiful history
$ git undo        # Undo last commit, keep changes
```

```
$ git unstage app.py    # Remove from staging
```

■ What's Next?

You now have a complete foundation in Git and GitHub. The path to mastery is practice. Here are the next steps to continue growing:

- **Practice every day:** Make real commits on real projects.
- **Contribute to open source:** Work with strangers' codebases.
- **Learn GitOps:** Infrastructure-as-code managed through Git.
- **Explore GitHub Actions deeply:** Build complex CI/CD pipelines.
- **Study git internals further:** Read the Pro Git book (free at git-scm.com).
- **Learn git-filter-repo:** For advanced history manipulation.
- **Explore Conventional Commits tooling:** commitizen, semantic-release.

■ Tip

The best way to learn Git is to break things in a safe environment and recover them. Create a test repository and intentionally make mistakes — bad merges, wrong resets, detached HEAD states — then practice recovering. Fear of Git commands disappears the moment you realize reflog almost always has your back.

Git & GitHub Complete Guide · From Beginner to Professional · git commit -m 'keep growing'